ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор		Ю.А. Скобцов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

Эволюционный метод прогнозирования стоимости программных проектов

по курсу: ЭВОЛЮЦИОННЫЕ МЕТОДЫ ПРОЕКТИРОВАНИЯ ПРОГРАММНО-ИНФОРМАЦИОННЫХ СИСТЕМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Теоретические сведения:

Модель COCOMO (Constructive Cost Model) - математическая модель оценки стоимости программных проектов. Используется зависимость трудозатрат от размера кода: Effort = a × (KLOC)^b, где a и b - калибровочные параметры.

Выполненная работа:

Вещественное кодирование. Каждое решение представляется вектором из трех вещественных чисел: [a, b, c] для расширенной модели Effort = a × (L)^b × (Me)^c.

Генетические операторы

- Отбор: Ранговый отбор - особи сортируются по фитнессу, вероятность выбора пропорциональна рангу
- Кроссовер: Арифметический - потомок = α×родитель1 + (1-α)×родитель2
- Мутация: Арифметическая - добавление случайного смещения к параметрам

Формирование выборок

Случайный отбор 13 проектов из 18 для обучения, остальные 5 - для тестирования. Индексы проектов выбираются случайно.

Фитнесс-функция

MMRE (Mean Magnitude of Relative Error): средняя величина относительной ошибки. Фитнесс = 1/(1 + MMRE).

Параметры ГА

- Мощность популяции: 50 особей
- Количество поколений: 100
- Вероятность мутации: 0.1 (10%)
- Кроссовер: применяется ко всем родительским парам

Входные данные:

Таблица 8.1. Экспериментальные данные проектов НАСА

Номер проекта	L	Me	E_f	E_{fm}	E_{fm2}
1	90,2000	30,0000	115,8000	124,8585	134,0202
2	46,2000	20,0000	96,0000	74,8467	84,1616
3	46,5000	19,0000	79,0000	75,4852	85,0112
4	54,5000	20,0000	909,8000	85,4349	94,9828
5	31,1000	35,0000	39,6000	50,5815	56,6580
6	67,5000	29,000	98,4000	99,0504	107,2609
7	12,8000	26,000	18,9000	24,1480	32,6461
8	10,5000	34,0000	10,3000	18,0105	25,0755
9	21,5000	31,0000	28,5000	37,2724	44,3086
10	3,1000	26,000	7,0000	4,5849	14,4563
11	4,2000	19,0000	9,0000	8,9384	19,9759
12	7,8000	31,0000	7,3000	13,5926	21,5763
13	2,1000	28,0000	5,0000	1,5100	11,2703
14	5,0000	29,0000	8,4000	8,2544	17,0887
15	78,6000	35,0000	98,7000	110,5249	118,0378
16	9,7000	27,0000	15,6000	18,2559	26,8312
17	12,5000	27,0000	23,9000	23,3690	31,6864
18	100,8000	34,0000	138,3000	135,4825	144,4587

Листинг программы, её выводы и графики:

```
import numpy as np
import matplotlib.pyplot as plt
import random
```

```python
from typing import List, Tuple, Callable
import pandas as pd

class COCOMOData:
    """Класс для работы с данными COCOMO NASA из таблицы 8.1"""
    def __init__(self):
        # Данные из таблицы 8.1 NASA
        # L - размер (KLOC), Me - методика, Ef - фактические трудозатраты
        self.data = [
            # [L (KLOC), Me, Ef, Efm, Efm2]
            [90.2, 30.0, 115.8, 124.8585, 134.0202],
            [46.2, 20.0, 96.0, 74.8467, 84.1616],
            [46.5, 19.0, 79.0, 75.4852, 85.0112],
            [54.5, 20.0, 909.8, 85.4349, 94.9828],
            [31.1, 35.0, 39.6, 50.5815, 56.6580],
            [67.5, 29.0, 98.4, 99.0504, 107.2609],
            [12.8, 26.0, 18.9, 24.1480, 32.6461],
            [10.5, 34.0, 10.3, 18.0105, 25.0755],
            [21.5, 31.0, 28.5, 37.2724, 44.3086],
            [3.1, 26.0, 7.0, 4.5849, 14.4563],
            [4.2, 19.0, 9.0, 8.9384, 19.9759],
            [7.8, 31.0, 7.3, 13.5926, 21.5763],
            [2.1, 28.0, 5.0, 1.5100, 11.2703],
            [5.0, 29.0, 8.4, 8.2544, 17.0887],
            [78.6, 35.0, 98.7, 110.5249, 118.0378],
            [9.7, 27.0, 15.6, 18.2559, 26.8312],
            [12.5, 27.0, 23.9, 23.3690, 31.6864],
            [100.8, 34.0, 138.3, 135.4825, 144.4587]
        ]

        self.column_names = ['L_KLOC', 'Me', 'Ef_actual', 'Efm', 'Efm2']

    def get_dataframe(self) -> pd.DataFrame:
        """Возвращает данные в виде DataFrame"""
        return pd.DataFrame(self.data, columns=self.column_names)

    def get_training_set(self, n_projects: int = 13) -> Tuple[np.ndarray, np.ndarray,
np.ndarray]:
        """Возвращает обучающее множество"""
        if n_projects > len(self.data):
            n_projects = len(self.data)

        # Выбираем случайные проекты для обучения
        indices = random.sample(range(len(self.data)), n_projects)
        training_data = [self.data[i] for i in indices]

        L_KLOC = np.array([item[0] for item in training_data])
        Me = np.array([item[1] for item in training_data])
        actual_effort = np.array([item[2] for item in training_data])

        return L_KLOC, Me, actual_effort, indices
```

```
    def get_test_set(self, training_indices: List[int]) -> Tuple[np.ndarray,
np.ndarray, np.ndarray]:
        """Возвращает тестовое множество на основе индексов обучения"""
        test_indices = [i for i in range(len(self.data)) if i not in training_indices]
        test_data = [self.data[i] for i in test_indices]

        if not test_data:
            return np.array([]), np.array([]), np.array([]), []

        L_KLOC = np.array([item[0] for item in test_data])
        Me = np.array([item[1] for item in test_data])
        actual_effort = np.array([item[2] for item in test_data])

        return L_KLOC, Me, actual_effort, test_indices

class GeneticAlgorithmCOCOMO:
    """Генетический алгоритм для настройки параметров COCOMO"""

    def __init__(self, population_size: int = 50, generations: int = 100):
        self.population_size = population_size
        self.generations = generations
        self.cocomo_data = COCOMOData()

        # Получаем обучающую выборку (13 проектов)
        self.L_train, self.Me_train, self.actual_effort_train, self.training_indices =
\
            self.cocomo_data.get_training_set(13)

        # Параметры расширенной модели COCOMO: Effort = a * (L)^b * (Me)^c
        self.param_bounds = {
            'a': (0.1, 10.0),
            'b': (0.8, 1.5),
            'c': (-0.5, 0.5)  # Влияние методики
        }

    def cocomo_model(self, params: np.ndarray, L: np.ndarray, Me: np.ndarray = None) -
> np.ndarray:
        """Расширенная модель COCOMO: Effort = a * (L)^b * (Me)^c"""
        a, b, c = params

        if Me is None:
            # Базовая модель: Effort = a * (L)^b
            return a * (L ** b)
        else:
            # Расширенная модель с учетом методики
            return a * (L ** b) * (Me ** c)

    def fitness_mmre(self, params: np.ndarray) -> float:
        """Фитнесс-функция: Mean Magnitude of Relative Error (MMRE)"""
        predicted = self.cocomo_model(params, self.L_train, self.Me_train)
        mre = np.abs(predicted - self.actual_effort_train) / self.actual_effort_train
        mmre = np.mean(mre)
        # Преобразуем в фитнесс (чем меньше MMRE, тем лучше)
```

```python
        return 1.0 / (1.0 + mmre)

    def initialize_population(self) -> np.ndarray:
        """Инициализация популяции - вещественные векторы"""
        population = np.zeros((self.population_size, 3))
        for i in range(self.population_size):
            population[i, 0] = random.uniform(*self.param_bounds['a'])
            population[i, 1] = random.uniform(*self.param_bounds['b'])
            population[i, 2] = random.uniform(*self.param_bounds['c'])
        return population

    def arithmetic_crossover(self, parent1: np.ndarray, parent2: np.ndarray) ->
Tuple[np.ndarray, np.ndarray]:
        """Арифметический кроссовер"""
        alpha = random.random()
        child1 = alpha * parent1 + (1 - alpha) * parent2
        child2 = alpha * parent2 + (1 - alpha) * parent1
        return child1, child2

    def arithmetic_mutation(self, individual: np.ndarray, mutation_rate: float = 0.1)
-> np.ndarray:
        """Арифметическая мутация"""
        mutated = individual.copy()
        for i in range(len(mutated)):
            if random.random() < mutation_rate:
                if i == 0:  # параметр a
                    mutated[i] += random.uniform(-0.5, 0.5)
                    mutated[i] = np.clip(mutated[i], *self.param_bounds['a'])
                elif i == 1:  # параметр b
                    mutated[i] += random.uniform(-0.1, 0.1)
                    mutated[i] = np.clip(mutated[i], *self.param_bounds['b'])
                else:  # параметр c
                    mutated[i] += random.uniform(-0.05, 0.05)
                    mutated[i] = np.clip(mutated[i], *self.param_bounds['c'])
        return mutated

    def rank_selection(self, population: np.ndarray, fitness: np.ndarray) ->
List[np.ndarray]:
        """Ранговый отбор"""
        # Сортируем по фитнессу (от лучшего к худшему)
        sorted_indices = np.argsort(fitness)[::-1]
        ranked_population = population[sorted_indices]

        # Вероятности выбора на основе ранга
        ranks = np.arange(len(population), 0, -1)
        probabilities = ranks / np.sum(ranks)

        selected = []
        for _ in range(self.population_size):
            selected_idx = np.random.choice(len(population), p=probabilities)
            selected.append(ranked_population[selected_idx])

        return selected
```

```python
    def run(self) -> Tuple[np.ndarray, List[float], List[float]]:
        """Запуск генетического алгоритма"""
        population = self.initialize_population()
        best_fitness_history = []
        avg_fitness_history = []
        best_mmre_history = []

        print("Запуск генетического алгоритма для COCOMO...")
        print("=" * 70)
        print(f"Обучающая выборка: {len(self.training_indices)} проектов")
        print(f"Индексы проектов: {sorted(self.training_indices)}")

        for generation in range(self.generations):
            # Оценка фитнесса
            fitness = np.array([self.fitness_mmre(ind) for ind in population])

            # Сохранение истории
            best_fitness = np.max(fitness)
            avg_fitness = np.mean(fitness)
            best_fitness_history.append(best_fitness)
            avg_fitness_history.append(avg_fitness)

            best_idx = np.argmax(fitness)
            best_params = population[best_idx]
            best_mmre = 1.0 / best_fitness - 1.0
            best_mmre_history.append(best_mmre)

            # Вывод прогресса
            if generation % 20 == 0 or generation == self.generations - 1:
                print(f"Поколение {generation:3d}: MMRE = {best_mmre:.4f}, "
                      f"a={best_params[0]:.3f}, b={best_params[1]:.3f},
c={best_params[2]:.3f}")

            # Отбор
            selected = self.rank_selection(population, fitness)

            # Кроссовер и мутация
            new_population = []
            for i in range(0, len(selected), 2):
                if i + 1 < len(selected):
                    parent1, parent2 = selected[i], selected[i + 1]
                    child1, child2 = self.arithmetic_crossover(parent1, parent2)
                    new_population.extend([child1, child2])
                else:
                    new_population.append(selected[i])

            # Мутация
            for i in range(len(new_population)):
                new_population[i] = self.arithmetic_mutation(new_population[i])

            population = np.array(new_population)
```

```
        # Лучшее решение
        final_fitness = np.array([self.fitness_mmre(ind) for ind in population])
        best_idx = np.argmax(final_fitness)
        best_solution = population[best_idx]
        best_mmre = 1.0 / final_fitness[best_idx] - 1.0

        print("=" * 70)
        print("ФИНАЛЬНЫЕ РЕЗУЛЬТАТЫ:")
        print(f"Лучшие параметры модели: a = {best_solution[0]:.4f}, b =
{best_solution[1]:.4f}, c = {best_solution[2]:.4f}")
        print(f"Лучший MMRE на обучающей выборке: {best_mmre:.4f}")

        return best_solution, best_fitness_history, avg_fitness_history,
best_mmre_history

def evaluate_model(best_params: np.ndarray, training_indices: List[int]):
    """Оценка модели на тестовой выборке"""
    cocomo_data = COCOMOData()
    L_test, Me_test, actual_effort_test, test_indices =
cocomo_data.get_test_set(training_indices)

    ga = GeneticAlgorithmCOCOMO()

    # Предсказания на обучающей выборке
    L_train, Me_train, actual_effort_train, _ = cocomo_data.get_training_set(13)
    L_train, Me_train, actual_effort_train, _ = cocomo_data.get_training_set(13)
    predicted_train = ga.cocomo_model(best_params, L_train, Me_train)
    mre_train = np.abs(predicted_train - actual_effort_train) / actual_effort_train
    mmre_train = np.mean(mre_train)

    print("\nОЦЕНКА КАЧЕСТВА МОДЕЛИ:")
    print("=" * 50)
    print("ОБУЧАЮЩАЯ ВЫБОРКА:")
    print(f"Количество проектов: {len(L_train)}")
    print(f"MMRE: {mmre_train:.4f}")

    # Предсказания на тестовой выборке
    if len(L_test) > 0:
        predicted_test = ga.cocomo_model(best_params, L_test, Me_test)
        mre_test = np.abs(predicted_test - actual_effort_test) / actual_effort_test
        mmre_test = np.mean(mre_test)
        print(f"\nТЕСТОВАЯ ВЫБОРКА:")
        print(f"Количество проектов: {len(L_test)}")
        print(f"Индексы проектов: {test_indices}")
        print(f"MMRE: {mmre_test:.4f}")

    return predicted_train, predicted_test, test_indices

def plot_results(best_params: np.ndarray, best_fitness_history: List[float],
                avg_fitness_history: List[float], best_mmre_history: List[float],
                training_indices: List[int]):
    """Построение графиков результатов"""
    cocomo_data = COCOMOData()
```

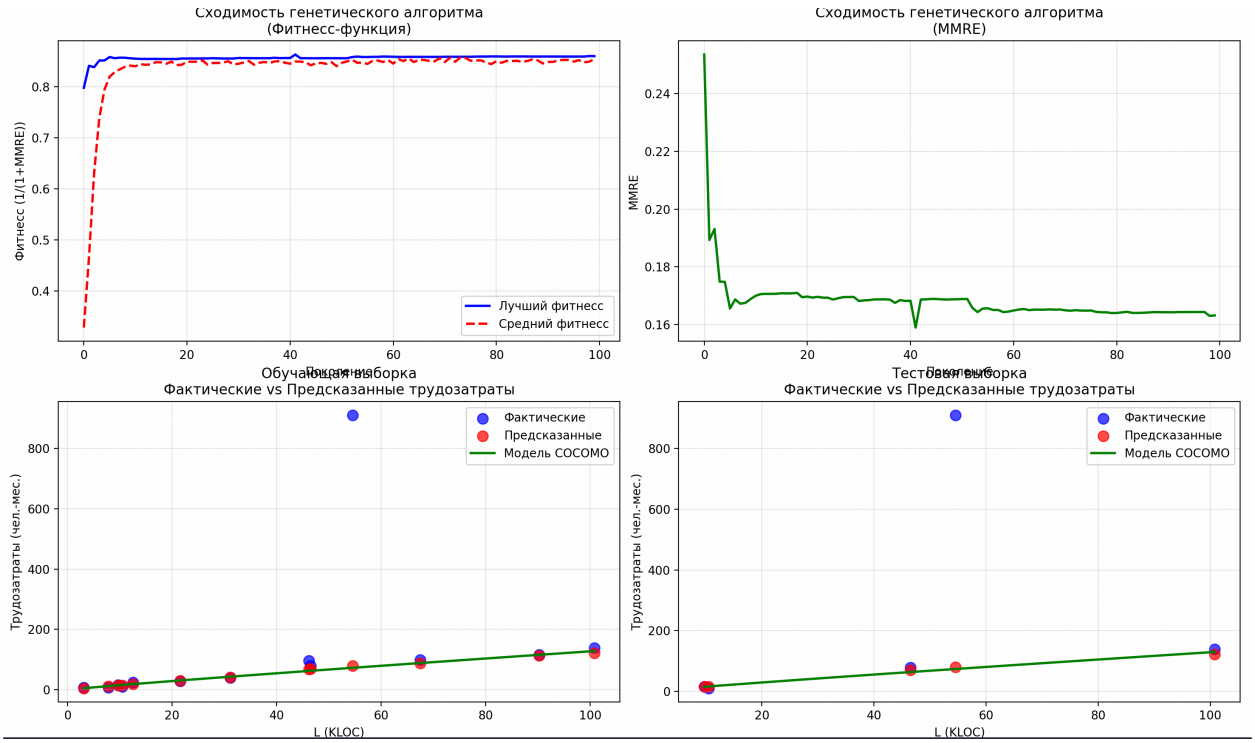
```
    df = cocomo_data.get_dataframe()

    L_train, Me_train, actual_effort_train, _ = cocomo_data.get_training_set(13)
    L_train, Me_train, actual_effort_train, _ = cocomo_data.get_training_set(13)
    L_test, Me_test, actual_effort_test, test_indices =
cocomo_data.get_test_set(training_indices)

    ga = GeneticAlgorithmCOCOMO()

    # График 1: Сходимость генетического алгоритма
    plt.figure(figsize=(16, 12))

    plt.subplot(2, 2, 1)
    plt.plot(best_fitness_history, 'b-', label='Лучший фитнесс', linewidth=2)
    plt.plot(avg_fitness_history, 'r--', label='Средний фитнесс', linewidth=2)
    plt.xlabel('Поколение')
    plt.ylabel('Фитнесс (1/(1+MMRE))')
    plt.title('Сходимость генетического алгоритма\n(Фитнесс-функция)')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # График 2: MMRE по поколениям
    plt.subplot(2, 2, 2)
    plt.plot(best_mmre_history, 'g-', linewidth=2)
    plt.xlabel('Поколение')
    plt.ylabel('MMRE')
    plt.title('Сходимость генетического алгоритма\n(MMRE)')
    plt.grid(True, alpha=0.3)

    # График 3: Фактические vs Предсказанные значения (обучающая выборка)
    plt.subplot(2, 2, 3)
    predicted_train = ga.cocomo_model(best_params, L_train, Me_train)

    plt.scatter(L_train, actual_effort_train, color='blue', label='Фактические',
alpha=0.7, s=80)
    plt.scatter(L_train, predicted_train, color='red', label='Предсказанные',
alpha=0.7, s=80)

    # Линия тренда
    L_sorted = np.sort(L_train)
    Me_avg = np.mean(Me_train)
    effort_sorted = ga.cocomo_model(best_params, L_sorted, np.full_like(L_sorted,
Me_avg))
    plt.plot(L_sorted, effort_sorted, 'g-', linewidth=2, label='Модель COCOMO')

    plt.xlabel('L (KLOC)')
    plt.ylabel('Трудозатраты (чел.-мес.)')
    plt.title('Обучающая выборка\nФактические vs Предсказанные трудозатраты')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # График 4: Тестовая выборка
    plt.subplot(2, 2, 4)
```

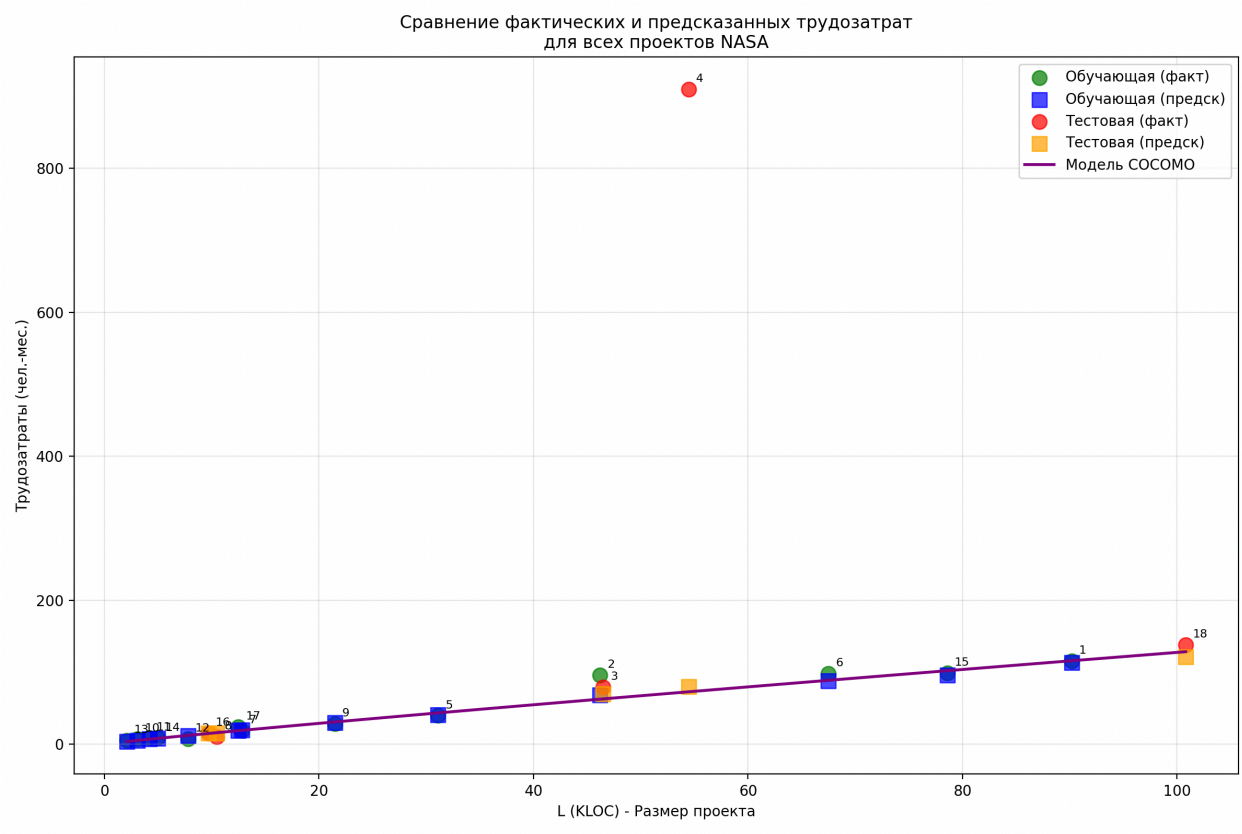
```
    if len(L_test) > 0:
        predicted_test = ga.cocomo_model(best_params, L_test, Me_test)
        plt.scatter(L_test, actual_effort_test, color='blue', label='Фактические',
alpha=0.7, s=80)
        plt.scatter(L_test, predicted_test, color='red', label='Предсказанные',
alpha=0.7, s=80)

        L_range = np.linspace(min(L_test), max(L_test), 100)
        effort_range = ga.cocomo_model(best_params, L_range, np.full_like(L_range,
np.mean(Me_test)))
        plt.plot(L_range, effort_range, 'g-', linewidth=2, label='Модель COCOMO')

        plt.xlabel('L (KLOC)')
        plt.ylabel('Трудозатраты (чел.-мес.)')
        plt.title('Тестовая выборка\nФактические vs Предсказанные трудозатраты')
        plt.legend()
    else:
        plt.text(0.5, 0.5, 'Тестовая выборка\nне доступна',
                horizontalalignment='center', verticalalignment='center',
                transform=plt.gca().transAxes, fontsize=12)
        plt.title('Тестовая выборка')

    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    # График 5: Сравнение всех проектов
    plt.figure(figsize=(12, 8))

    all_L = df['L_KLOC'].values
    all_actual = df['Ef_actual'].values
    all_Me = df['Me'].values
    all_predicted = ga.cocomo_model(best_params, all_L, all_Me)

    # Разделяем на обучающую и тестовую выборки
    train_mask = np.isin(range(len(all_L)), training_indices)

    plt.scatter(all_L[train_mask], all_actual[train_mask],
               color='green', label='Обучающая (факт)', alpha=0.7, s=100, marker='o')
    plt.scatter(all_L[train_mask], all_predicted[train_mask],
               color='blue', label='Обучающая (предск)', alpha=0.7, s=100, marker='s')

    if len(test_indices) > 0:
        test_mask = np.isin(range(len(all_L)), test_indices)
        plt.scatter(all_L[test_mask], all_actual[test_mask],
                   color='red', label='Тестовая (факт)', alpha=0.7, s=100, marker='o')
        plt.scatter(all_L[test_mask], all_predicted[test_mask],
                   color='orange', label='Тестовая (предск)', alpha=0.7, s=100,
marker='s')

    # Добавляем номера проектов
    for i, (x, y) in enumerate(zip(all_L, all_actual)):
```

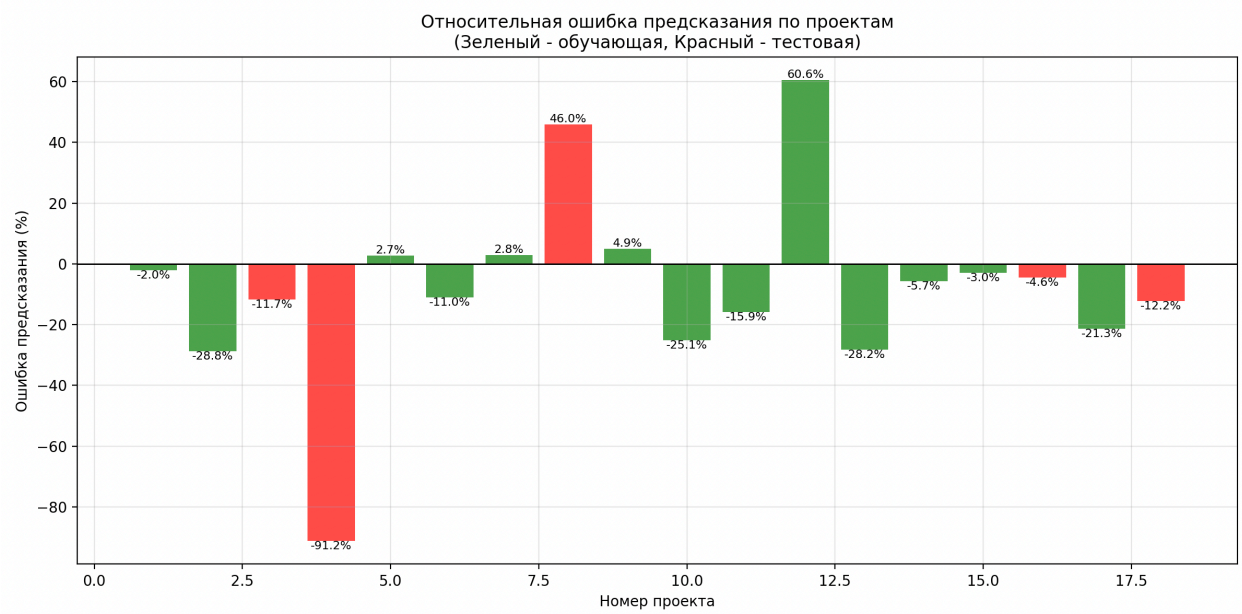
```
        plt.annotate(f'{i+1}', (x, y), xytext=(5, 5), textcoords='offset points',
fontsize=8)

    L_range = np.linspace(min(all_L), max(all_L), 100)
    effort_range = ga.cocomo_model(best_params, L_range, np.full_like(L_range,
np.mean(all_Me)))
    plt.plot(L_range, effort_range, 'purple', linewidth=2, label='Модель COCOMO')

    plt.xlabel('L (KLOC) - Размер проекта')
    plt.ylabel('Трудозатраты (чел.-мес.)')
    plt.title('Сравнение фактических и предсказанных трудозатрат\nдля всех проектов
NASA')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    # График 6: Ошибки предсказания
    plt.figure(figsize=(12, 6))

    errors = (all_predicted - all_actual) / all_actual * 100

    colors = ['green' if i in training_indices else 'red' for i in range(len(errors))]

    bars = plt.bar(range(1, len(errors) + 1), errors, color=colors, alpha=0.7)
    plt.axhline(y=0, color='black', linestyle='-', linewidth=1)
    plt.xlabel('Номер проекта')
    plt.ylabel('Ошибка предсказания (%)')
    plt.title('Относительная ошибка предсказания по проектам\n(Зеленый - обучающая,
Красный - тестовая)')

    # Добавляем подписи
    for i, (bar, error) in enumerate(zip(bars, errors)):
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height,
                f'{error:.1f}%', ha='center', va='bottom' if height >= 0 else 'top',
fontsize=8)

    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def print_detailed_comparison(best_params: np.ndarray, training_indices: List[int]):
    """Вывод детального сравнения предсказаний"""
    cocomo_data = COCOMOData()
    df = cocomo_data.get_dataframe()

    ga = GeneticAlgorithmCOCOMO()

    all_L = df['L_KLOC'].values
    all_Me = df['Me'].values
    all_actual = df['Ef_actual'].values
    all_predicted = ga.cocomo_model(best_params, all_L, all_Me)
```

```
    errors = (all_predicted - all_actual) / all_actual * 100

    print("\nДЕТАЛЬНОЕ СРАВНЕНИЕ ПРЕДСКАЗАНИЙ:")
    print("=" * 90)
    print(f"{'Проект':<8} {'L (KLOC)':<10} {'Me':<8} {'Факт.':<10} {'Предск.':<10}
{'Ошибка %':<10} {'Тип':<10}")
    print("-" * 90)

    for i in range(len(df)):
        project_type = "Обучающий" if i in training_indices else "Тестовый"
        print(f"{i+1:<8} {all_L[i]:<10.1f} {all_Me[i]:<8.1f} {all_actual[i]:<10.1f} "
              f"{all_predicted[i]:<10.1f} {errors[i]:<10.1f} {project_type:<10}")

    print("-" * 90)

    # Статистика по ошибкам
    train_errors = errors[training_indices]
    test_indices = [i for i in range(len(df)) if i not in training_indices]

    if test_indices:
        test_errors = errors[test_indices]
        print(f"\nСТАТИСТИКА ОШИБОК:")
        print(f"Обучающая выборка: MMRE = {np.mean(np.abs(train_errors)/100):.4f}")
        print(f"Тестовая выборка:  MMRE = {np.mean(np.abs(test_errors)/100):.4f}")
        print(f"Общая MMRE: {np.mean(np.abs(errors)/100):.4f}")

def main():
    """Основная функция"""
    print("Вариант 2:")
    print("  - Тип: Генетический алгоритм")
    print("  - Кодирование: Вещественный вектор")
    print("  - Фитнесс-функция: MMRE (Mean Magnitude of Relative Error)")
    print("  - Кроссовер: Арифметический")
    print("  - Мутация: Арифметическая")
    print("  - Репродукция: Ранговый отбор")
    print("=" * 70)

    # Запуск генетического алгоритма
    ga = GeneticAlgorithmCOCOMO(population_size=50, generations=100)
    best_params, best_fitness_history, avg_fitness_history, best_mmre_history =
ga.run()

    # Оценка модели
    predicted_train, predicted_test, test_indices = evaluate_model(best_params,
ga.training_indices)

    # Детальное сравнение
    print_detailed_comparison(best_params, ga.training_indices)

    # Построение графиков
    plot_results(best_params, best_fitness_history, avg_fitness_history,
                 best_mmre_history, ga.training_indices)
```

```
    # Дополнительная информация
    print("\nИНФОРМАЦИЯ О МОДЕЛИ:")
    print("=" * 50)
    print("Расширенная модель COCOMO: Effort = a × (L)^b × (Me)^c")
    print(f"Найденные параметры: a = {best_params[0]:.4f}, b = {best_params[1]:.4f}, c
= {best_params[2]:.4f}")

    # Пример предсказания для нового проекта
    print(f"\nПРИМЕР ПРЕДСКАЗАНИЯ:")
    test_L = 50.0
    test_Me = 25.0
    predicted_effort = ga.cocomo_model(best_params, np.array([test_L]),
np.array([test_Me]))[0]
    print(f"Проект: L = {test_L} KLOC, Me = {test_Me}")
    print(f"Предсказанные трудозатраты: {predicted_effort:.1f} чел.-мес.")

if __name__ == "__main__":
    main()
```




Сравнение фактических и предсказанных трудозатрат
для всех проектов NASA
Обучающая (факт)
Обучающая (предск)
Тестовая (факт)
Тестовая (предск)
Модель COCOMO
Трудозатраты (чел.-мес.)
L (KLOC) - Размер проекта


Относительная ошибка предсказания по проектам
(Зеленый - обучающая, Красный - тестовая)
Ошибка предсказания (%)
Номер проекта
-2.0%
-28.8%
-11.7%
-91.2%
2.7%
-11.0%
2.8%
46.0%
4.9%
-25.1%
-15.9%
60.6%
-28.2%
-5.7%
-3.0%
-4.6%
-21.3%
-12.2%

```
Вариант 2:
  - Тип: Генетический алгоритм
  - Кодирование: Вещественный вектор
  - Фитнесс-функция: MMRE (Mean Magnitude of Relative Error)
  - Кроссовер: Арифметический
  - Мутация: Арифметическая
  - Репродукция: Ранговый отбор
======================================================================
Запуск генетического алгоритма для COCOMO...
======================================================================
Обучающая выборка: 13 проектов
Индексы проектов: [0, 1, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14, 16]
Поколение   0: MMRE = 0.2535, a=3.519, b=1.145, c=-0.432
Поколение  20: MMRE = 0.1697, a=4.029, b=0.953, c=-0.275
Поколение  40: MMRE = 0.1682, a=4.200, b=0.960, c=-0.290
Поколение  60: MMRE = 0.1648, a=4.335, b=0.947, c=-0.285
Поколение  80: MMRE = 0.1640, a=4.368, b=0.943, c=-0.280
Поколение  99: MMRE = 0.1632, a=4.356, b=0.939, c=-0.277
======================================================================
ФИНАЛЬНЫЕ РЕЗУЛЬТАТЫ:
Лучшие параметры модели: a = 4.5130, b = 0.9236, c = -0.2745
Лучший MMRE на обучающей выборке: 0.1631

ОЦЕНКА КАЧЕСТВА МОДЕЛИ:
==================================================
ОБУЧАЮЩАЯ ВЫБОРКА:
Количество проектов: 13
MMRE: 0.2412

ТЕСТОВАЯ ВЫБОРКА:
Количество проектов: 5
Индексы проектов: [2, 3, 7, 15, 17]
MMRE: 0.3314
```

```
ДЕТАЛЬНОЕ СРАВНЕНИЕ ПРЕДСКАЗАНИЙ:
==================================================================================================
Проект   L (KLOC)   Me       Факт.      Предск.    Ошибка %   Тип
--------------------------------------------------------------------------------------------------
1        90.2       30.0     115.8      113.4      -2.0       Обучающий
2        46.2       20.0     96.0       68.4       -28.8      Обучающий
3        46.5       19.0     79.0       69.7       -11.7      Тестовый
4        54.5       20.0     909.8      79.6       -91.2      Тестовый
5        31.1       35.0     39.6       40.7       2.7        Обучающий
6        67.5       29.0     98.4       87.6       -11.0      Обучающий
7        12.8       26.0     18.9       19.4       2.8        Обучающий
8        10.5       34.0     10.3       15.0       46.0       Тестовый
9        21.5       31.0     28.5       29.9       4.9        Обучающий
10       3.1        26.0     7.0        5.2        -25.1      Обучающий
11       4.2        19.0     9.0        7.6        -15.9      Обучающий
12       7.8        31.0     7.3        11.7       60.6       Обучающий
13       2.1        28.0     5.0        3.6        -28.2      Обучающий
14       5.0        29.0     8.4        7.9        -5.7       Обучающий
15       78.6       35.0     98.7       95.8       -3.0       Обучающий
16       9.7        27.0     15.6       14.9       -4.6       Тестовый
17       12.5       27.0     23.9       18.8       -21.3      Обучающий
18       100.8      34.0     138.3      121.5      -12.2      Тестовый
--------------------------------------------------------------------------------------------------

СТАТИСТИКА ОШИБОК:
Обучающая выборка: MMRE = 0.1631
Тестовая выборка:  MMRE = 0.3314
Общая MMRE: 0.2099

ИНФОРМАЦИЯ О МОДЕЛИ:
==================================================
Расширенная модель COCOMO: Effort = a × (L)^b × (Me)^c
Найденные параметры: a = 4.5130, b = 0.9236, c = -0.2745

ПРИМЕР ПРЕДСКАЗАНИЯ:
Проект: L = 50.0 KLOC, Me = 25.0
Предсказанные трудозатраты: 69.2 чел.-мес.
```

Вывод:

В результате проделанной работы я научился применять генетический алгоритм с вещественным кодированием для настройки параметров модели COCOMO, освоил использование арифметических операторов кроссовера и мутации, а также рангового отбора, реализовал расчет фитнесс-функции на основе MMRE для оценки точности предсказания трудозатрат программных проектов, получил практический опыт разделения исходных данных на обучающую и тестовую выборки и анализа результатов оптимизации с помощью визуализации, что позволило успешно решить задачу калибровки модели оценки стоимости программного обеспечения на реальных данных проектов НАСА.